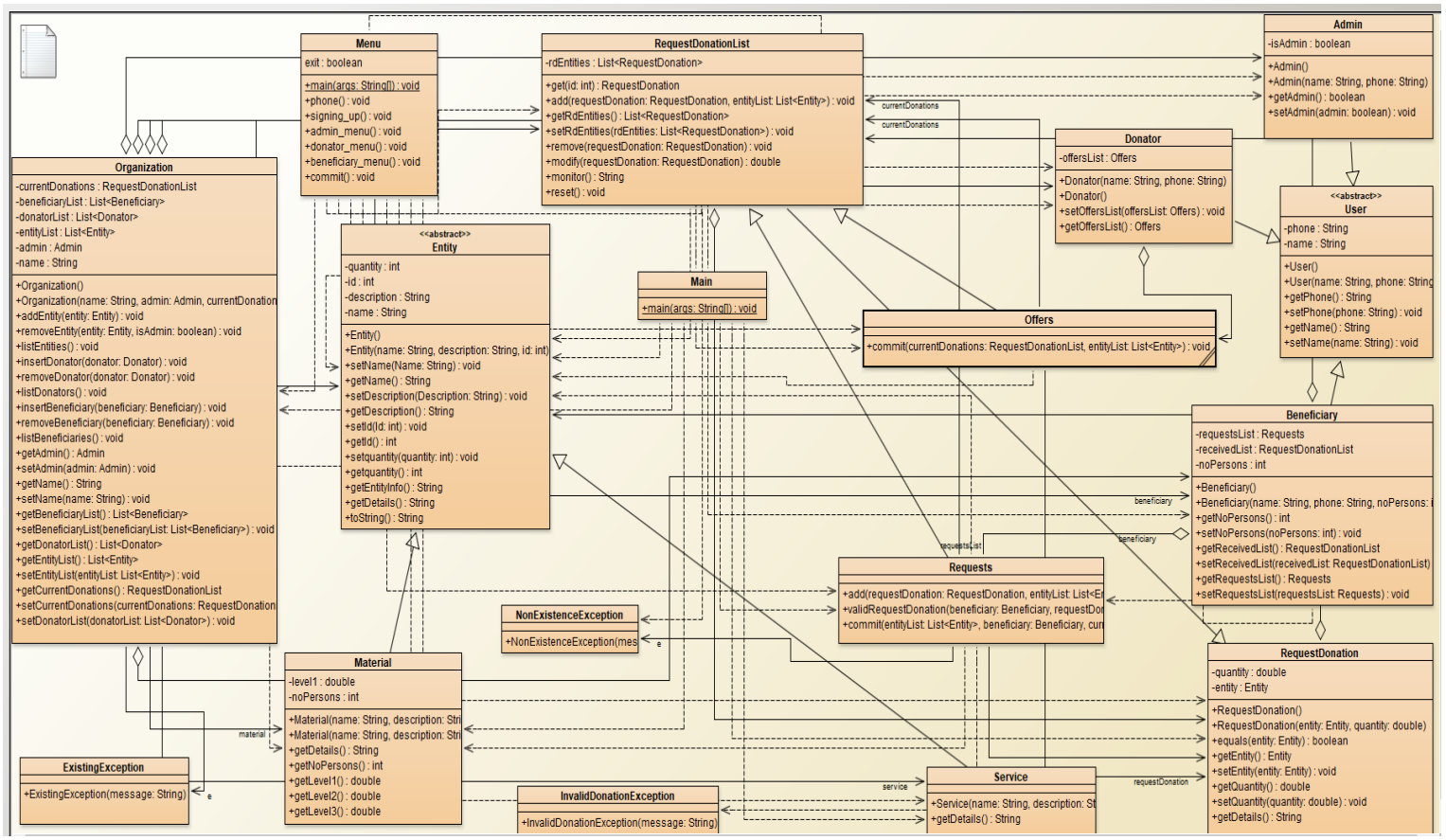
ΕΡΓΑΣΙΑ ΟΝΤΟΚΕΝΤΡΙΚΟΥ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ 2021-2022

Τα μέλη της ομάδας:

- Ρόδη Ασημίνα (ΑΜ: 1059693 και e-mail: st1059693@ceid.upatras.gr)
- Μπίρμπας Παναγιώτης (ΑΜ: 1069365 και e-mail: st1069365@ceid.upatras.gr)
- Σίψας Ανδρέας (ΑΜ: 1080395 και e-mail: st1080395@ceid.upatras.gr)

Link για το code repository: https://github.com/jazzr99/ontok22

UML διάγραμμα:



Περιγραφή του προγράμματος και και παρουσίαση των κλάσεων:

- **user:** Περιέχει δύο private μεταβλητές τύπου string, την name και την phone. Χρησιμοποιούμε getters και setters για να έχουμε πρόσβαση στα πεδία και να δημιουργούμε κάθε φορά καινούργια αντικείμενα για κάθε χρήστη τηρώντας έτσι την αρχή της ενθυλάκωσης.
- **entity:** Περιλαμβάνει το όνομα (string name), την περιγραφή (string description) και τον κωδικό του είδους (int id). Διαθέτει μεθόδους για την επιστροφή των πληροφοριών που προαναφέρθηκαν και την @overide η οποία εμφανίζει τις πληροφορίες του προϊόντος όλες μαζί για τις υποκλάσσεις της (material,service) καθώς και την public abstract string getDetails την οποία υλοποιούν οι υποκλάσεις της.
- **admin:** Είναι υποκλάση της user, κάνει την χρήση μιας private boolean μεταβλητής(isAdmin) καθώς και των name και phone, που είναι super, για να μπορεί με την χρήση getters setters να ελέγχει με βάση τον αριθμό τηλεφώνου την σύνδεση στον λογαριασμό admin.

- **donator:** Είναι υποκλάση της user και χρησιμοποιεί τις μεταβλητές super name και phone. Οι μέθοδοι που χρησιμοποιεί αλληλεπιδρούν με αυτές της offer και έτσι ενημερώνεται η OffersList διατηρώντας την αρχή της ενθυλάκωσης.
- **beneficiary:** Είναι υποκλάση της user, χρησιμοποιεί τις μεταβλητές super name και phone ώστε να κρατάει για τον κάθε beneficiary στις ανάλογες λίστες(receivedList και requestsList) των αντίστοιχων κλάσεων(RequestDonationList και Requests) τον αριθμό των ατόμων της οικογένειας, τα είδη που αιτείται να πάρει και τα είδη που πρέπει να πάρει.
- **ExistingException:** Ως υποκλάση την RunTimeException διαχειρίζεται τις εξαιρέσεις που που δημιουργούνται στην κλάση organization όταν δεν πληρούνται οι προϋποθέσεις των μεθόδων addEntity, insertDonator και insertBeneficiary, δηλαδή όταν υπάρχουν ήδη στο σύστημα καταγεγραμμένα αντικείμενα με το ίδιο όνομα.
- **InvalidDonationException:** Ως υποκλάση την RunTimeException διαχειρίζεται τις εξαιρέσεις που που δημιουργούνται στην κλάση Requests όταν δεν πληρούνται οι προϋποθέσεις της μεθόδου validRequestDonation.
- **main:** Περιέχει τα δεδομένα για τα έξι Entities. Τα τρια Materials είναι τα Milk, Sugar και Rice με συγκεκριμένες ποσότητες ανά level και περιγραφή προϊόντος. Αναλυτικότερα το Milk με κωδικό προϊόντος 1,στο level 1 έχει 2.0 τεμάχια, στο level 2 έχει 4.0 και στο level 3 έχει 10.0 και αντιστοίχως για τα υπόλοιπα materials. Από την άλλη τα services περιέχουν μόνο περιγραφή υπηρεσίας και προσωπικού κωδικού. Τα τρία Services είναι Medical Support, Nursery Support και Baby Sitting. Στη συνέχεια εισάγονται οι τέσσερις χρήστες, δηλαδή ο admin Aegon με αριθμό τηλεφώνου 26102210, οι beneficiaries Visenya και Rhaenys με αριθμούς 2102210 και 22340221 αντίστοιχα. Τέλος περιέχει τα request των δύο ωφελούμενων και τις προσφορές του δωρητή.
- **material:** Είναι υποκλάση της entity και υλοποιώντας την getDetails (λόγω της κληρονομικότητας) εμφανίζει για το ανάλογο material τις αντίστοιχες πληροφορίες μέσω των gets.
- **menu:** Αρχικά στην main εμφανίζει χαιρετισμό στον χρήστη και ρωτάει αν έχει υπάρχοντα λογαριασμό. Αν έχει λογαριασμό τον πηγαίνει στην μέθοδο phone αλλιώς στη μέθοδο signing_up. Η phone ελέγχει αν ο χρήστης έχει ήδη λογαριασμό μέσω του αριθμού του τηλεφώνου του και τον κατευθύνει στην αντίστοιχη μέθοδο(Admin/Beneficiary/Donator). Η signing_up ρωτάει τον χρήστη αν θέλει να εγγραφεί ως beneficiary ή ως donator. Ανάλογα με την απάντηση δημιουργεί προφίλ με όνομα χρήστη, αριθμό τηλεφώνου και αριθμό μελών οικογενείας.

Αν ο χρήστης είναι ο **admin** τότε καλείται η **admin_menu** και εμφανίζονται οι προσωπικές πληροφορίες του χρήστη όπως όνομα, τηλέφωνο και το ότι είναι ο διαχειριστής. Έπειτα ζητάει από τον χρήστη να επιλέξει μια ενέργεια για να συνεχίσει. Αρχικά με την view επιλέγει κατηγορία(material ή service) και στην συνέχεια ένα από τα entities ώστε να του εμφανιστεί τελικά μια πλήρης περιγραφή των χαρακτηριστικών της. Ύστερα στο monitor organization τον ρωτάει να επιλέξει από τρεις επιλογές(η λίστα με τους beneficiaries, η λίστα με τους donators και η επιλογή να κάνει reset τις λίστες των beneficiaries). Σαν τρίτη επιλογή του mo είναι η επιλογή back που τον επιστρέφει στην επιλογή εισαγωγής αριθμού. Τέταρτη επιλογή είναι να αποσυνδεθεί και τον ρωτάει αν θέλει να κάνει πάλι σύνδεση χρήστη. Έπειτα τον οδηγεί στην μέθοδο phone. Τέλος η πέμπτη επιλογή του mo είναι η έξοδος από το σύστημα.

Αν ο χρήστης είναι **donator** τότε καλείται η donator_menu και δίνει την επιλογή να εισάγει δωρεά, να δει τις ήδη υπάρχουσες δωρεές του, να πάει στην προηγούμενη ενέργειά του, να αποσυνδεθεί και τέλος να βγει από το σύστημα. Στην πρώτη ενέργεια επιλέγει κατηγορία(materials-services) και έπειτα ένα συγκεκριμένο entity. Όταν επιλεγεί ένα από αυτά εμφανίζονται οι πληροφορίες του και η ερώτηση αν θέλει να κάνει δωρεά για αυτό το προϊόν. Αν επιλέξει show offers τότε μέσω της organization εμφανίζονται οι προσφορές που έχει ήδη κάνει. Επιλογή 3 πηγαίνει πίσω χρησιμοποιώντας την μέθοδο phone. Με την επιλογή 4 του εμφανίζεται το αντιστοιχο μήνυμα και καλείται πάλι η μέθοδος phone, Στην επιλογή 5 μέσω της boolean μεταβλητής exit βγαίνει από το σύστημα.

Αν ο χρήστης είναι **beneficiary** τότε καλείται η beneficiary_menu που έχει τις επιλογές προσθήκη αιτήματος, εμφάνιση αιτημάτων, επιστροφή, αποσύνδεση και έξοδος. Με την πρώτη επιλογή διαλέγει material ή service και έπειτα ένα συγκεκριμένο entity. Του εμφανίζονται οι

λεπτομέρειες για το entity αυτό και η ερώτηση πόση ποσότητα επιθυμείτε να παραλάβετε. Μέσω της κλάσης Requests δημιουργείται το αίτημα του και καλείται η μέθοδος commit. Στην δεύτερη επιλογή εμφανίζονται οι λίστες με όσα έχουν παραλάβει οι beneficiaries. Αντίστοιχα τα υπόλοιπα όπως στο donator_menu. Η μέθοδος commit δεν καλείται στο αρχικό μενού του beneficiary αλλά βρίσκεται ως επιλογή κατά το τελικό στάδιο δημιουργίας ενός αιτήματος του beneficiary. Εμφανίζει ερώτηση στον χρήστη εάν θέλει να προσθέσει την αίτηση και του εμφανίζει ανάλογη απάντηση.

- **NonExistanceException:** Ως υποκλάση την RunTimeException διαχειρίζεται τις εξαιρέσεις που που δημιουργούνται στην κλάση Requests όταν κατά την κλήση της μεθόδου commit εντοπίζεται ότι το entity που ζητείται δεν προσφέρεται από τον οργανισμό.
- **offers:** Είναι υποκλάση της RequestDonationList και μέσω της λίστας CurrentDonations δείχνει το σύνολο των ειδών που θέλει να προσφέρει ο Donator. Επίσης έχει την δυνατότητα να διαγράψει όλα τα είδη που περιέχουν οι προσφορές.
- **organization:** Ορίζονται οι κατηγορίες των προσφερόμενων entities, αρχικοποιούνται οι λίστες που τα περιέχουν καθώς οι λίστες με τους beneficiaries και τους donators. Επειδή οι παραπάνω λίστες είναι private για να μπορέσουν οι μέθοδοι να επεξεργαστούν τα δεδομένα τους γίνεται χρήση getters-setters. Δεν κάναμε χρήση wrappers για την currentDonations.
- **RequestDonation:** Ελέγχει τα id των entities μέσω μιας boolean μεθόδου (equals) και παίρνει τα κατάλληλα δεδομένα μέσω των getters-setters. Επιστρέφει μέσω του getDetails το είδος και την ποσότητα του entity που αιτείται.
- **RequestDonationList:** Είναι υποκλάση της RequestDonation. Ο constructor δημιουργεί αντικείμενα στη λίστα που είναι τα αιτήματα και οι αντίστοιχες μέθοδοι ελέγχουν την λίστα.
- **Requests:** Πρακτικά ελέγχει όλα τα αιτήματα των beneficiary με τις 3 μεθόδους της και επιστρέφει τα ανάλογα μηνύματα με τη χρήση των public class που αφορουν στις εξαιρέσεις (NonExistanceException, InvalidDonationException)
- **service:** Είναι υποκλάση της entity, κάνει χρήση των super μεταβλητών name, description και id ώστε να ξεχωρίζει τα services και υλοποιώντας την getdetails επιστρέφει σαν πληροφορία ότι το αντικείμενο είναι service.

Κώδικας:

```
public class Admin extends User{

    private boolean isAdmin = true;

    public Admin(){}

    public Admin(String name, String phone){

        super(name, phone);

    }


    //Getters-Setters

    public boolean getAdmin(){

        return isAdmin;

    }
```

```
    public void setAdmin(boolean admin){
        isAdmin=admin;
    }}

import java.util.ArrayList;
import java.util.List;

public class Beneficiary extends User{
    private int noPersons=1;
    private RequestDonationList receivedList;
    private Requests requestsList;
    public Beneficiary(){}

    public Beneficiary(String name, String phone, int noPersons){
        super(name, phone);
        this.noPersons=noPersons;
    }
    // GETTERS-SETTERS
    public int getNoPersons(){
        return noPersons;
    }
    public void setNoPersons(int noPersons){
        this.noPersons=noPersons;
    }
    public RequestDonationList getReceivedList(){
        return receivedList;
    }
    public void setReceivedList(RequestDonationList receivedList){
```

```
        this.receivedList=receivedList;
    }
    public Requests getRequestsList(){
        return requestsList;
    }
    public void setRequestsList(Requests requestsList){
        this.requestsList=requestsList;
    }}
public class Donator extends User{
    private Offers offersList;
    public Donator(String name, String phone){
        super(name, phone);
    }
    // GETTER-SETTER
    public void setOffersList(Offers offersList){
    this.offersList = offersList;
    }
    public Offers getOffersList(){
    return this.offersList;
    }
    public Donator(){}
}
public abstract class Entity {
    private String name;
    private String description;
    private int id;
    public Entity() {
    }
```

```
public Entity(String name, String description, int id){
    this.name=name;
    this.description=description;
    this.id=id;
}
// GETTERS-SETTERS
public void setName(String Name){
this.name=Name;
}
public String getName(){
return name;
}
public void setDescription(String Description){
this.description=Description;
}

public String getDescription(){
return description;
}
public void setId(int Id){
this.id=Id;
}
public int getId(){
return id;
}
public String getEntityInfo(){
    return("The current donation is: " + this.name + " \n Description: " + this.description + " \n id: " + this.id
        + "." );
}
```

```
    public abstract String getDetails();

    @Override
    public String toString() {
        return getEntityInfo() + getDetails();
    }public class ExistingException extends RuntimeException{
    public ExistingException(String message){
        super(message);
    }}
}
public class InvalidDonationException extends RuntimeException{
    public InvalidDonationException(String message){
        super(message);
    }}import java.util.ArrayList;
import java.util.List;

public class Main{
    public static void main(String[]args){
        List<Entity> entityList = new ArrayList<>();

        Organization organization=new Organization();

        //Insert data for Materials.

        Material milk = new Material("Milk","Long life cow milk. (1.5L)", 1, 2.0, 4.0, 10.0);

        Material sugar = new Material("Sugar","White granulated sugar. (500g)", 2, 1.0, 4.0, 8.0);
```

```
Material rice = new Material("Rice","Basmati rice. (250g)", 3, 2.0, 4.0, 8.0);

//Insert data for Services.

Service MedicalSupport = new Service("MedicalSupport","Provided by experts in Famiy Medicine.",4);

Service NurserySupport = new Service("NurserySupport","Provided by certified nursery care
    givers.",5);

Service BabySitting = new Service("BabySitting","Provided by Primary Education students.",6);



entityList.add(milk);

entityList.add(sugar);

entityList.add(rice);

entityList.add(MedicalSupport);

entityList.add(NurserySupport);

entityList.add(BabySitting);



//Insert data for Users.

Admin admin = new Admin("Aegon", "26102210");
```

```
Beneficiary beneficiary01 = new Beneficiary("Visenya", "2102210",3);

Beneficiary beneficiary02 = new Beneficiary("Rhaenys", "22340221",2);

Donator donator01 = new Donator("Maegor", "22810221");



Requests requestsBe01 = new Requests();

/* Benefiaciary01 asks for rice */

RequestDonation requestFromBeneficiary01_1 = new RequestDonation(rice,3);

requestsBe01.add(requestFromBeneficiary01_1, entityList, beneficiary01);

/* Benefiaciary01 asks for milk */

RequestDonation requestFromBeneficiary01_2 = new RequestDonation(milk,1);

requestsBe01.add(requestFromBeneficiary01_2, entityList, beneficiary01);

/* Benefiaciary01 asks for Medical Support */

RequestDonation requestFromBeneficiary01_3 = new RequestDonation(MedicalSupport,3);

requestsBe01.add(requestFromBeneficiary01_3, entityList, beneficiary01);
```

```
        Offers offerDo01 = new Offers();

        /* Donator offers Nursery Support*/

        RequestDonation requestForfDonator01_1 = new RequestDonation(NurserySupport, 5);

        offerDo01.add(requestForfDonator01_1, entityList);

        /* Donator offers Medical Support*/

        RequestDonation requestForfDonator01_2 = new RequestDonation(MedicalSupport, 4);

        offerDo01.add(requestForfDonator01_2, entityList);

    }
}
public class Material extends Entity{
    private int noPersons;
    private double level1,level2,level3;
    public Material(String name, String description, int id){
        super(name, description, id);
    }
    public Material(String name, String description, int id, double level1, double level2, double level3){
    super(name, description, id);
    this.level1=level1;
    this.level2=level2;
    this.level3=level3;
    }
```

```
    public String getDetails(){

        return("Level-1 is: " + this.getLevel1() + "\n Level-2 is: " + this.getLevel2() + "\n Level-3 is: " +
          this.getLevel3() + "\n It's a Material. ");

      }

    public int getNoPersons (){

        return noPersons;

      }

    public double getLevel1 (){

        return level1;

      }

    public double getLevel2 (){

        return level2;

      }

    public double getLevel3 (){

        return level3;

      } }

public class NonExistenceException extends RuntimeException{


    public NonExistenceException(String message) {

      super(message);

    }

}import java.util.List;

import java.util.ArrayList;


public class Offers extends RequestDonationList {

    public void commit(RequestDonationList currentDonations, List<Entity> entityList){

      for(RequestDonation requestDonation: currentDonations.getRdEntities()){

        currentDonations.add(requestDonation, entityList);

       }
```

```
        currentDonations.getRdEntities().removeAll(currentDonations.getRdEntities());
    }}
import java.util.ArrayList;
import java.util.List;



public class Organization {

    private String name;
    private Admin admin;
    private List<Entity> entityList = new ArrayList<>();
    private List<Donator> donatorList = new ArrayList<>();
    private List<Beneficiary> beneficiaryList = new ArrayList<>();
    private RequestDonationList currentDonations;

    public Organization() {


    }

    public Organization(String name, Admin admin, RequestDonationList currentDonations) {
        this.name = name;
        this.admin = admin;
        this.currentDonations = currentDonations;
    }

    //METHODS
    public void addEntity(Entity entity) {
        try {
```

```
        this.entityList.add(entity);
    }catch(ExistingException e) {
        System.out.println(e);
    }
}

public void removeEntity(Entity entity, boolean isAdmin) {
     if(isAdmin){
         this.entityList.remove(entity);}
}
 public void listEntities() {
    List<Material> materialsList = new ArrayList<>();
    List<Service> servicesList = new ArrayList<>();
    System.out.println("There are two possible catagories to choose from: Material or Services");

    for(Entity entity : this.entityList) {
        if(entity instanceof Material) {
            materialsList.add((Material) entity);
        }
        else {
            servicesList.add((Service) entity);
        }
    }

    for(Material material : materialsList) {
        material.getDetails();
    }
```

```
    for(Service service : servicesList) {
        service.getDetails();
    }
}
public void insertDonator(Donator donator) {
    try {
        this.donatorList.add(donator);
    }
    catch(ExistingException e) {
        System.out.println(e);
    }
}



public void removeDonator(Donator donator) {
    this.donatorList.remove(donator);
}
public void listDonators() {
    for(Donator donator : this.donatorList) {
        System.out.println(donator.getName() + donator.getPhone());
    }
}
public void insertBeneficiary(Beneficiary beneficiary) {
    try {
        this.beneficiaryList.add(beneficiary);
    }
    catch(ExistingException e) {
        System.out.println(e);
```

```
    }
}

public void removeBeneficiary(Beneficiary beneficiary) {
    this.beneficiaryList.remove(beneficiary);
}




public void listBeneficiaries() {
    for(Beneficiary beneficiary : this.beneficiaryList) {

        System.out.println(beneficiary.getReceivedList().monitor());

    }
}



// GETTERS-SETTERS
public Admin getAdmin() {
    return admin;
}

public void setAdmin (Admin admin) {
    this.admin=admin;
}
```

```
 public String getName() {
     return name;
 }

 public void setName(String name) {
     this.name=name;
 }


 public List<Beneficiary> getBeneficiaryList() {
     return beneficiaryList;
 }

 public void setBeneficiaryList(List<Beneficiary> beneficiaryList) {
     this.beneficiaryList=beneficiaryList;
 }

 public List<Donator> getDonatorList() {
     return donatorList;
 }
 public List<Entity> getEntityList() {
     return entityList;
 }

 public void setEntityList(List<Entity> entityList) {
     this.entityList=entityList;
 }
public RequestDonationList getCurrentDonations() {
```

```
        return currentDonations;
    }

    public void setCurrentDonations(RequestDonationList currentDonations) {
        this.currentDonations=currentDonations;
    }

    public void setDonatorList(List<Donator> donatorList) {
        this.donatorList=donatorList;
    }
}
 import java.util.*;

class RequestDonation
{
    private Entity entity;
    private double quantity;

    public RequestDonation (){ }

    public RequestDonation(Entity entity, double quantity){
     this.entity=entity;
     this.quantity=quantity;
    }
    public boolean equals(Entity entity) {
        if (this.getEntity().getId() == entity.getId()){
            return true;
        }
```

```
     return false;
    }
    // GETTERS-SETTERS
    public Entity getEntity() {
        return entity;
    }
    public void setEntity(Entity entity) {
        this.entity=entity;
    }
    public double getQuantity() {
        return quantity;
    }
    public void setQuantity(double quantity) {
        this.quantity=quantity;
    }
    public String getDetails()
    {return ("RequestDonation:Entity:"+entity+ " Quantity:"+quantity);}
}
import java.util.List;
import java.util.ArrayList;

public class RequestDonationList extends RequestDonation
{
    private List<RequestDonation> rdEntities=new ArrayList<>();
    public RequestDonation get(int id)
    {
       RequestDonation requestDonation=new RequestDonation();
       for (RequestDonation rd:rdEntities){
```

```
        if (rd.getEntity().getId()==id){
            requestDonation=rd;
        }}
    return requestDonation;
}
public void add(RequestDonation requestDonation, List<Entity> entityList)
{
    if(!entityList.contains(requestDonation.getEntity())) {
        throw new NonExistenceException("This entity does not exist in the organization!");
    }
    for (RequestDonation donation:rdEntities){
            if(requestDonation.equals(donation.getEntity())){
                donation.setQuantity(donation.getQuantity() + requestDonation.getQuantity());
            }
             else{
                rdEntities.add(requestDonation);
            }}}
public List<RequestDonation> getRdEntities(){
    return rdEntities;
}
public void setRdEntities(List<RequestDonation> rdEntities){
    this.rdEntities=rdEntities;
}
public void remove(RequestDonation requestDonation)
{
    rdEntities.remove(requestDonation);
}
public double modify(RequestDonation requestDonation)
```

```
    {
        return requestDonation.getQuantity();
    }
    public String monitor()
    {
        for (int n=0; n<rdEntities.size(); n++)
    {
        return rdEntities.get(n).getDetails();
    }
        return ".";
    }
    public void reset()
    {
        rdEntities.removeAll(rdEntities);
    }}
import java.util.Iterator;
import java.util.List;

public class Requests extends RequestDonationList {

    public void add(RequestDonation requestDonation, List<Entity> entityList, Beneficiary beneficiary){
        try{
            if(!validRequestDonation(beneficiary,requestDonation)){
                throw new InvalidDonationException("Sorry, this donation is invalid.");
            }
            super.add(requestDonation, entityList);
        }
        catch(NonExistenceException e)
```

```
    { e.getMessage();
    } }
public boolean validRequestDonation(Beneficiary beneficiary, RequestDonation requestDonation){
    double level=0;
    double totalQuantity=0;
    if (requestDonation.getEntity() instanceof Material){
        if(beneficiary.getNoPersons()==1){
            level=((Material) requestDonation.getEntity()).getLevel1();
        }
        else if(beneficiary.getNoPersons()>=2 && beneficiary.getNoPersons()<=4){
            level=((Material) requestDonation.getEntity()).getLevel2();
        }
        else if(beneficiary.getNoPersons()>=5){
            level=((Material) requestDonation.getEntity()).getLevel3();
        }
        else{
            System.out.println("This is not acceptable.\nPlease try again.");
        } }
    for(RequestDonation receivedMaterial : beneficiary.getReceivedList().getRdEntities()){
        if(((Material) requestDonation.getEntity()).equals((Material) receivedMaterial.getEntity())){
            totalQuantity += receivedMaterial.getQuantity();
        } }
    totalQuantity += requestDonation.getQuantity();
    if(totalQuantity>level){
        return false;
    }
    return true;
}
```

```
  public void commit(List<Entity> entityList, Beneficiary beneficiary, RequestDonationList
          currentDonations){


     Iterator iterator=getRdEntities().iterator();

     while(iterator.hasNext()){

        RequestDonation requestDonationIterator=(RequestDonation) iterator.next();

        if(!entityList.contains(requestDonationIterator.getEntity())) {

           throw new NonExistenceException("This entity does not exist in the organization!");

        }

        if(!validRequestDonation(beneficiary,requestDonationIterator)){

           throw new InvalidDonationException("Sorry, this donation is invalid.");

        }

        getRdEntities().remove(requestDonationIterator);


          currentDonations.get(requestDonationIterator.getEntity().getId()).setQuantity(currentDonations.get
          Quantity() - requestDonationIterator.getQuantity());

        beneficiary.getReceivedList().getRdEntities().add(requestDonationIterator);

     } } }
public class Service extends Entity{


  public Service(String name,String description,int id){

     super(name,description,id);

  }

  public String getDetails(){

     return("This donation is a Service");

  }}public abstract class User {

  private String name;

  private String phone;


  public User(){ }
```

```
    public User(String name, String phone){
     this.name=new String(name);
     this.phone=new String(phone);
    }
    // GETTERS-SETTERS
    public String getPhone(){
        return phone;
    }
    public void setPhone(String phone){
        this.phone=phone;
    }
    public String getName(){
        return name;
    }
    public void setName(String name) {
        this.name=name;
    }}
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.io.*;


public class Menu
{
boolean exit;
 public static void main(String[] args)
  {
```

```
    Menu menu = new Menu();

    System.out.println("Welcome!\nDo you have an existing account? (1=yes / 2=no)");

      Scanner scanner1 = new Scanner(System.in);
      Integer scanner_1 = scanner1.nextInt();

    if(scanner_1 == 1)
     {
     menu.phone();
     }
    else
    {
     menu.signing_up();
    }
  }

//Checking if User already exists.
public void phone()
  {
      System.out.println("Please enter your phone number:");

      Scanner scannumber = new Scanner(System.in);
      Integer scan_number = scannumber.nextInt();

      if(scan_number==26102210)
      {System.out.println("Hello Aegon");
          admin_menu();
```

```
        }
        else if(scan_number==2102210)
        {System.out.println("Hello Visenya");
            beneficiary_menu();
        }
        else if(scan_number==22340221)
        {System.out.println("Hello Rhaenys");
             beneficiary_menu();
        }
        else if(scan_number==22810221)
        {System.out.println("Hello Maegor");
             donator_menu();
        }
    }
// Signing up form
public void signing_up()
 {
        System.out.println("Would you like to become Beneficiary or Donator? (1=beneficiary /2=donator)");

        Scanner scanChoice = new Scanner(System.in);
        Integer scan_choice = scanChoice.nextInt();

        if(scan_choice == 1)
        {
            Beneficiary newBeneficiary = new Beneficiary("NewBeneficiary","Number",1);
            System.out.println("Name: " + newBeneficiary.getName());
            System.out.println("Phone: " + newBeneficiary.getPhone());
            System.out.println("NoPersons: " + newBeneficiary.getNoPersons());
```

```
            System.out.println("----------------------------");


            beneficiary_menu();
        }
        else if(scan_choice == 2)
        {
            Donator newDonator = new Donator("NewDonator","Number");
            System.out.println("Name: " + newDonator.getName());
            System.out.println("Phone: " + newDonator.getPhone());
            System.out.println("----------------------------");


            donator_menu();
        }
    }
// Enter the Admin menu
 public void admin_menu()
{
    System.out.println("Your Info:");
    Admin admin = new Admin("Aegon", "26102210");
    System.out.println("Name: " + admin.getName() + "\nPhone: " + admin.getPhone() + "\nAdmin: " +
            admin.getAdmin() );
    System.out.println("[View(v)/Monitor Organization(mo)/Back(b)/Logout(l)/Exit(e)]");


    Scanner scanAdminChoice = new Scanner(System.in);
    String scanAdmin_choice = scanAdminChoice.nextLine();


    Boolean exit;
    switch(scanAdmin_choice)
    {
```

```
case "v": System.out.println("You choce View.");
System.out.println("Select a category:");
System.out.println("1.Material");
System.out.println("2.Services");
System.out.println("Press 1 or 2.");

Scanner category_number = new Scanner(System.in);
Integer numberCategory = category_number.nextInt();

if(numberCategory == 1)
 {
   System.out.println("These are the materials:");
   System.out.println("1.milk");
   System.out.println("2.sugar");
   System.out.println("3.rice");
   System.out.println("Press 1 or 2 or 3.");

   Scanner material_number = new Scanner(System.in);
   Integer materialNumber = material_number.nextInt();

   if(materialNumber == 1)
    {
      Material milk = new Material("Milk","Long life cow milk. (1.5L)", 1, 2.0, 4.0, 10.0);
      System.out.println("Milk");
      System.out.println("Description: " + milk.getDescription());
      System.out.println("ID: " + milk.getId());
      System.out.println(milk.getDetails());
    }
```

```
  else if(materialNumber == 2)
   {
     Material sugar = new Material("Sugar","White granulated sugar. (500g)", 2, 1.0, 4.0, 8.0);
     System.out.println("Sugar");
     System.out.println("Description: " + sugar.getDescription());
     System.out.println("ID: " + sugar.getId());
     System.out.println(sugar.getDetails());
   }
  else if(materialNumber == 3)
   {
     Material rice = new Material("Rice","Basmati rice. (250g)", 3, 2.0, 4.0, 8.0);
     System.out.println("Rice");
     System.out.println("Description: " + rice.getDescription());
     System.out.println("ID: " + rice.getId());
     System.out.println(rice.getDetails());
   }
 }
else if(numberCategory == 2)
 {
  System.out.println("These are the Services:");
  System.out.println("1.MedicalSupport");
  System.out.println("2.NurserySupport");
  System.out.println("3.Babysitting");
  System.out.println("Press 1 or 2 or 3.");

  Scanner service_number = new Scanner(System.in);
  Integer serviceNumber = service_number.nextInt();
```

```
  if(serviceNumber == 1)
   {
     Service MedicalSupport = new Service("MedicalSupport","Provided by experts in Famiy
        Medicine.",4);
     System.out.println("MedicalSupport");
     System.out.println("Description: " + MedicalSupport.getDescription());
     System.out.println("ID: " + MedicalSupport.getId());
   }
  else if(serviceNumber == 2)
   {
     Service NurserySupport = new Service("NurserySupport","Provided by certified nursery care
        givers.",5);
     System.out.println("NurserySupport");
     System.out.println("Description: " + NurserySupport.getDescription());
     System.out.println("ID: " + NurserySupport.getId());
   }
  else if(serviceNumber == 3)
   {
     Service BabySitting = new Service("BabySitting","Provided by Primary Education students.",6);
     System.out.println("Babysitting");
     System.out.println("Description: " + BabySitting.getDescription());
     System.out.println("ID: " + BabySitting.getId());
   }
 }
break;

case "mo": System.out.println("You choce Monitor Organization:");
System.out.println("Press one of the following options:");
System.out.println("1.List Beneficiaries");
```

```
System.out.println("2.List Donators");
System.out.println("3.Reset Beneficiaries Lists");
System.out.println("Press 1 or 2 or 3.");

Scanner scan_monitor = new Scanner(System.in);
Integer scanMonitor = scan_monitor.nextInt();

if(scanMonitor == 1)
{
 System.out.println("Beneficiaries: ");
 Beneficiary beneficiary01 = new Beneficiary("Visenya", "2102210",3);
 Beneficiary beneficiary02 = new Beneficiary("Rhaenys", "22340221",2);
 Organization organization = new Organization();
 organization.listBeneficiaries();
 System.out.println("1.beneficiary01");
 System.out.println("2.beneficiary02");
 System.out.println("Press 1 or 2.");

Scanner scan_beneficiary = new Scanner(System.in);
Integer scanBeneficiary = scan_beneficiary.nextInt();

if(scanBeneficiary == 1)
 {
   System.out.println("beneficiary01: ");
   System.out.println("Name: " + beneficiary01.getName());
   System.out.println("Phone: " + beneficiary01.getPhone() );
   System.out.println("Number of family members: " + beneficiary01.getNoPersons() );
   System.out.println(beneficiary01.getReceivedList());
```

```
   System.out.println("Delete receivedList of beneficiary01?(y/n)");

   Scanner scan_delete1 = new Scanner(System.in);
   String scanDelete1 = scan_delete1.nextLine();

if(scanDelete1 == "y")
 {
   RequestDonationList requestDonationList = new RequestDonationList();
   requestDonationList.reset();
   System.out.println("ReceivedList deleted.");
 }
else if (scanDelete1 == "n")
 {
  System.out.println("The list was not deleted.");
 }

System.out.println("--------------------------");
System.out.println("Delete beneficiary01?(y/n)");

Scanner scan_delete2 = new Scanner(System.in);
String scanDelete2 = scan_delete2.nextLine();

if(scanDelete2 == "y")
 {
    organization.removeBeneficiary(beneficiary01);
    System.out.println("Beneficiary01 deleted");
 }
else if(scanDelete2 == "n")
```

```
 {
 System.out.println("Beneficiary01 was not deleted.");
 }
}

if(scanBeneficiary == 2)
{
 System.out.println("beneficiary02: ");
 System.out.println("Name: " + beneficiary02.getName());
 System.out.println("Phone: " + beneficiary02.getPhone() );
 System.out.println("Number of family members: " + beneficiary02.getNoPersons() );
 System.out.println(beneficiary02.getReceivedList());
 System.out.println("Delete receivedList of beneficiary02?(y/n)");

 Scanner scan_delete3 = new Scanner(System.in);
 String scanDelete3 = scan_delete3.nextLine();

if(scanDelete3 == "y")
 {
    RequestDonationList requestDonationList = new RequestDonationList();
    requestDonationList.reset();
    System.out.println("ReceivedList deleted");
 }
else if (scanDelete3 == "n")
{
  System.out.println("The list was not deleted.");
}
```

```
 System.out.println("--------------------------");
 System.out.println("Delete beneficiary01?(y/n)");

Scanner scan_delete4 = new Scanner(System.in);
String scanDelete4 = scan_delete4.nextLine();

if(scanDelete4 == "y")
{
    organization.removeBeneficiary(beneficiary02);
    System.out.println("Beneficiary02 deleted");
}
else if(scanDelete4 == "n")
 {
 System.out.println("Beneficiary02 was not deleted.");}
 }

}
else if(scanMonitor == 2)
{
    System.out.println("Donators:");
    Donator donator01 = new Donator("Maegor", "22810221");
    Organization organization =  new Organization();
    organization.listDonators();
    System.out.println("donator");

    donator01.getOffersList();

    System.out.println("Delete donator?(y/n)");
```

```
    Scanner scan_delete5 = new Scanner(System.in);
    String scanDelete5 = scan_delete5.nextLine();

    if(scanDelete5 == "y")
    {
        organization.removeDonator(donator01);
        System.out.println("Donator deleted");
    }
    else if(scanDelete5 == "n")
    {
        System.out.println("Donator was not deleted.");
    }
}
else if(scanMonitor == 3)
 {
    System.out.println("Reset Beneficiaries Lists");
    Organization organization = new Organization();
    for(Beneficiary beneficiary:organization.getBeneficiaryList())
    {
        beneficiary.getReceivedList().reset();
    }
    System.out.println("The action is complete.");
}
break;

case "b":
System.out.println("Back:");
```

```
    phone();
    break;

    case "l":
     System.out.println("You are disconnected.");
     System.out.println("Do you want to sign in?.");
     phone();
    break;

    case "e": System.out.println("Exit");
    exit = true;
    break;

 }
}
// Enter the Donator menu
    public void donator_menu()
    {
        System.out.println("Choose one option to continue:");
        System.out.println("1.Add Offers");
        System.out.println("2.Show Offers");
        System.out.println("3.Back");
        System.out.println("4.Logout");
        System.out.println("5.Exit");
        System.out.println("Choose 1 or 2 or 3 or 4 or 5");

        Scanner donChoice = new Scanner(System.in);
        int don_Choice  = donChoice.nextInt();
```

```
switch(don_Choice)
{
case 1 : System.out.println("Choose 1 or 2.");
System.out.println("1.Material or 2.Services");

Scanner categoryChoice1 = new Scanner(System.in);
int category_Choice1  = categoryChoice1.nextInt();

if(category_Choice1 == 1)
{
    Material milk = new Material("Milk","Long life cow milk. (1.5L)", 1, 2.0, 4.0, 10.0);
    Material sugar = new Material("Sugar","White granulated sugar. (500g)", 2, 1.0, 4.0, 8.0);
    Material rice = new Material("Rice","Basmati rice. (250g)", 3, 2.0, 4.0, 8.0);

    System.out.println("Choose 1 or 2 or 3.");
    System.out.println("1.Milk");
    System.out.println("2.Sugar");
    System.out.println("3.Rice");

    Scanner materialChoice = new Scanner(System.in);
    int material_Choice  = materialChoice.nextInt();

    if(material_Choice == 1)
    {
        System.out.println("Milk:");
        System.out.println(milk.getDetails());
```

```
    System.out.println("Would you like to donate milk? (y/n)");
    Scanner scanner_1 = new Scanner(System.in);
    String Scanner_1 = scanner_1.nextLine();
    if(Scanner_1 == "y"){
        List<Entity> entityList = new ArrayList<>();
        RequestDonation requestDonation01 = new RequestDonation(milk,1);
        Offers donatorOffers = new Offers();
        donatorOffers.add(requestDonation01,entityList);
    }
    else {
     break;
    }
        return;
}
else if(material_Choice == 2)
{
    System.out.println("Sugar:");
    System.out.println(sugar.getDetails());

    System.out.println("Would you like to donate Sugar? (y/n)");
    Scanner scanner_2 = new Scanner(System.in);
    String Scanner_2 = scanner_2.nextLine();
    if(Scanner_2 == "y"){
        List<Entity> entityList = new ArrayList<>();
        RequestDonation requestDonation02 = new RequestDonation(sugar,2);
        Offers donatorOffers = new Offers();
        donatorOffers.add(requestDonation02,entityList);
    }
```

```
        else {
         break;
        }
            return;
    }
    else if(material_Choice == 3)
    {
        System.out.println("Rice:");
        System.out.println(rice.getDetails());

        System.out.println("Would you like to donate rice? (y/n)");
        Scanner scanner_3 = new Scanner(System.in);
        String Scanner_3 = scanner_3.nextLine();
        if(Scanner_3 == "y"){
            List<Entity> entityList = new ArrayList<>();
            RequestDonation requestDonation03 = new RequestDonation(rice,3);
            Offers donatorOffers = new Offers();
            donatorOffers.add(requestDonation03,entityList);
        }
        else {
         break;
        }
            return;
    }

}
else if(category_Choice1 == 2)
{
```

```
Service MedicalSupport = new Service("MedicalSupport","Provided by experts in Famiy
  Medicine.",4);

Service NurserySupport = new Service("NurserySupport","Provided by certified nursery care
  givers.",5);

Service BabySitting = new Service("BabySitting","Provided by Primary Education students.",6);


System.out.println("1.MedicalSupprt");

System.out.println("2.NurserySupport");

System.out.println("3.BabySitting");

System.out.println("Choose 1 or 2 or 3.");


Scanner serviceChoice = new Scanner(System.in);

int service_Choice  = serviceChoice.nextInt();


if(service_Choice == 1)

{

    System.out.println("MedicalSupprt:");

    System.out.println(MedicalSupport.getDetails());


    System.out.println("Would you like to donate MedicalSupport? (y/n)");

    Scanner scanner_4 = new Scanner(System.in);

    String Scanner_4 = scanner_4.nextLine();

    if(Scanner_4 == "y"){

        List<Entity> entityList = new ArrayList<>();

        RequestDonation requestDonation04 = new RequestDonation(MedicalSupport,4);

        Offers donatorOffers = new Offers();

        donatorOffers.add(requestDonation04,entityList);

    }

    else {
```

```
     break;
    }
        return;
}
else if(service_Choice == 2)
{
    System.out.println("NurserySupport:");
    System.out.println(NurserySupport.getDetails());

    System.out.println("Would you like to donate NurserySupport? (y/n)");
    Scanner scanner_5 = new Scanner(System.in);
    String Scanner_5 = scanner_5.nextLine();
    if(Scanner_5 == "y"){
        List<Entity> entityList = new ArrayList<>();
        RequestDonation requestDonation05 = new RequestDonation(NurserySupport,5);
        Offers donatorOffers = new Offers();
        donatorOffers.add(requestDonation05,entityList);
    }
    else {
     break;
    }
        return;
}
else if(service_Choice == 3)
{
    System.out.println("BabySitting:");
    System.out.println(BabySitting.getDetails());
```

```
        System.out.println("Would you like to donate BabySitting? (y/n)");
        Scanner scanner_6 = new Scanner(System.in);
        String Scanner_6 = scanner_6.nextLine();
        if(Scanner_6 == "y"){
            List<Entity> entityList = new ArrayList<>();
            RequestDonation requestDonation06 = new RequestDonation(BabySitting,6);
            Offers donatorOffers = new Offers();
            donatorOffers.add(requestDonation06,entityList);
        }
        else {
         break;
        }
            return;
    }

}

case 2 :Donator donator01 = new Donator("Maegor", "22810221");

Organization organization =  new Organization();
System.out.println("Donator's Offers:");
System.out.println(organization.getCurrentDonations());

break;

case 3:
System.out.println("Back:");
phone();
```

```
    break;

    case 4:
     System.out.println("You are disconnected.");
     System.out.println("Do you want to sign in?.");
     phone();
    break;

    case 5: System.out.println("Exit");
    exit = true;
    break;
    }

}
// Enter the Beneficiary menu
  public void beneficiary_menu()
  {
    System.out.println("Choose one option to continue:");
    System.out.println("1.Add Request");
    System.out.println("2.Show Requests");
    System.out.println("3.Back");
    System.out.println("4.Logout");
    System.out.println("5.Exit");
    System.out.println("Choose 1 or 2 or 3 or 4 or 5");

    Scanner benChoice = new Scanner(System.in);
    int ben_Choice  = benChoice.nextInt();
```

```
switch(ben_Choice)
{
case 1 : System.out.println("Choose 1 or 2.");
System.out.println("1.Material or 2.Services");

Scanner categoryChoice2 = new Scanner(System.in);
int category_Choice2  = categoryChoice2.nextInt();

if (category_Choice2 == 1)
{
    System.out.println("Choose 1 or 2 or 3.");
    System.out.println("1.Milk");
    System.out.println("2.Sugar");
    System.out.println("3.Rice");

    Scanner materialChoice = new Scanner(System.in);
    int material_Choice  = materialChoice.nextInt();

    if(material_Choice == 1)
    {
        Material milk = new Material("Milk","Long life cow milk. (1.5L)", 1, 2.0, 4.0, 10.0);
        System.out.println("Milk:");
        System.out.println(milk.getDetails());
        System.out.println("Please enter quantity.");
        System.out.println("1. 2L or 2. 4L or 3. 10L");
        Scanner milkChoose = new Scanner(System.in);
        Integer milk_Choose = milkChoose.nextInt();
        if(milk_Choose == 1)
```

```
    {
    List<Entity> entityList = new ArrayList<>();
    Requests request1ofbeneficiary = new Requests();
    RequestDonation milkRequest = new RequestDonation(milk,1);
    request1ofbeneficiary.add(milkRequest,entityList);
    commit();
    }
    else if(milk_Choose == 2)
    {
    List<Entity> entityList = new ArrayList<>();
    Requests request2ofbeneficiary = new Requests();
    RequestDonation milkRequest = new RequestDonation(milk,2);
    request2ofbeneficiary.add(milkRequest,entityList);
    commit();
    }
    else if(milk_Choose == 3)
    {
    List<Entity> entityList = new ArrayList<>();
    Requests request3ofbeneficiary = new Requests();
    RequestDonation milkRequest = new RequestDonation(milk,3);
    request3ofbeneficiary.add(milkRequest,entityList);
    commit();
    }
}
else if(material_Choice == 2)
{
    Material sugar = new Material("Sugar","White granulated sugar. (500g)", 2, 1.0, 4.0, 8.0);
    System.out.println("Sugar:");
```

```
System.out.println(sugar.getDetails());
System.out.println("Please enter quantity.");
System.out.println("1. 1Kg or 2. 4KG or 3. 8Kg");
Scanner sugarChoose = new Scanner(System.in);
Integer sugar_Choose = sugarChoose.nextInt();
if(sugar_Choose == 1)
{
List<Entity> entityList = new ArrayList<>();
Requests request4ofbeneficiary = new Requests();
RequestDonation sugarRequest = new RequestDonation(sugar,1);
request4ofbeneficiary.add(sugarRequest,entityList);
commit();
}
else if(sugar_Choose == 2)
{
List<Entity> entityList = new ArrayList<>();
Requests request5ofbeneficiary = new Requests();
RequestDonation sugarRequest = new RequestDonation(sugar,2);
request5ofbeneficiary.add(sugarRequest,entityList);
commit();
}
else if(sugar_Choose == 3)
{
List<Entity> entityList = new ArrayList<>();
Requests request6ofbeneficiary = new Requests();
RequestDonation sugarRequest = new RequestDonation(sugar,3);
request6ofbeneficiary.add(sugarRequest,entityList);
commit();
```

```
    }
}
else if(material_Choice == 3)
{
    Material rice = new Material("Rice","Basmati rice. (250g)", 3, 2.0, 4.0, 8.0);
    System.out.println("Rice:");
    System.out.println(rice.getDetails());
    System.out.println("Please enter quantity.");
    System.out.println("1. 2Kg or 2. 4KG or 3. 8Kg");
    Scanner riceChoose = new Scanner(System.in);
    Integer rice_Choose = riceChoose.nextInt();
    if(rice_Choose == 1)
    {
    List<Entity> entityList = new ArrayList<>();
    Requests request7ofbeneficiary = new Requests();
    RequestDonation riceRequest = new RequestDonation(rice,1);
    request7ofbeneficiary.add(riceRequest,entityList);
    commit();
    }
    else if(rice_Choose == 2)
    {
    List<Entity> entityList = new ArrayList<>();
    Requests request8ofbeneficiary = new Requests();
    RequestDonation riceRequest = new RequestDonation(rice,2);
    request8ofbeneficiary.add(riceRequest,entityList);
    commit();
    }
    else if(rice_Choose == 3)
```

```
        {
        List<Entity> entityList = new ArrayList<>();
        Requests request9ofbeneficiary = new Requests();
        RequestDonation riceRequest = new RequestDonation(rice,3);
        request9ofbeneficiary.add(riceRequest,entityList);
        commit();
        }
    }
}
else if(category_Choice2 == 2)
{
    System.out.println("1.MedicalSupprt");
    System.out.println("2.NurserySupport");
    System.out.println("3.BabySitting");
    System.out.println("Choose 1 or 2 or 3.");

    Scanner serviceChoice = new Scanner(System.in);
    int service_Choice  = serviceChoice.nextInt();

    if(service_Choice == 1)
    {
      Service MedicalSupport = new Service("MedicalSupport","Provided by experts in Famiy
     Medicine.",4);
      System.out.println("MedicalSupport");
      System.out.println(MedicalSupport.getDetails());
      List<Entity> entityList = new ArrayList<>();
      Requests request10ofbeneficiary = new Requests();
      RequestDonation medicalSupportRequest = new RequestDonation(MedicalSupport,4);
      request10ofbeneficiary.add(medicalSupportRequest,entityList);
```

```
        commit();
    }
    else if(service_Choice == 2)
    {
        Service NurserySupport = new Service("NurserySupport","Provided by certified nursery care
      givers.",5);
        System.out.println("NurserySupport");
        System.out.println(NurserySupport.getDetails());
        List<Entity> entityList = new ArrayList<>();
        Requests request11ofbeneficiary = new Requests();
        RequestDonation NurserySupportRequest = new RequestDonation(NurserySupport,5);
        request11ofbeneficiary.add(NurserySupportRequest,entityList);
        commit();
    }
    else if(service_Choice == 3)
    {
        Service BabySitting = new Service("BabySitting","Provided by Primary Education students.",6);
        System.out.println("BabySitting");
        System.out.println(BabySitting.getDetails());
        List<Entity> entityList = new ArrayList<>();
        Requests request12ofbeneficiary = new Requests();
        RequestDonation BabySittingRequest = new RequestDonation(BabySitting,6);
        request12ofbeneficiary.add(BabySittingRequest,entityList);
        commit();
    }
    }
```

```
case 2 : Beneficiary beneficiary01 = new Beneficiary("Visenya", "2102210",3);
```

```
Beneficiary beneficiary02 = new Beneficiary("Rhaenys", "22340221",2);

Organization organization =  new Organization();
System.out.println("Beneficiary's 01 Requests:");
System.out.println(beneficiary01.getReceivedList());

System.out.println("Beneficiary's 02 Requests:");
System.out.println(beneficiary02.getReceivedList());

break;

case 3:
System.out.println("Back:");
phone();

break;

case 4:
 System.out.println("You are disconnected.");
 System.out.println("Do you want to sign in?.");
 phone();

break;

case 5: System.out.println("Exit");
exit = true;

break;
```

```
    }
    }

public void commit()
    {
    System.out.println("Would you like to add this request of donation? (y/n)");
    Scanner commitScanner = new Scanner(System.in);
    String commit_scanner = commitScanner.nextLine();
    if (commit_scanner=="y")
    {
    RequestDonationList currentDonations = new RequestDonationList();
    Beneficiary beneficiary = new Beneficiary();
    List<Entity> entityList = new ArrayList<>();
    Requests requests = new Requests();
    requests.commit(entityList,beneficiary,currentDonations);
    System.out.println("This request has been commited successfully!");
    }
    else if (commit_scanner=="n")
    {
    System.out.println("This request has not been commited.");
    phone();
    }
    }
}
```